9.5. Evaluation Metrics

Previous Section:

9.4. Outliers in Data

Contents:

After the dataset has been cleaned, transformed, and prepared, the next step is to train a machine learning model. Although model training is often considered a separate topic, understanding how to evaluate a trained model is an essential part of every data analytics workflow.

The choice of model depends on the prediction task.

- **Regression:** Linear Regression; Decision Tree Regressor; Random Forest Regressor
- **Classification:** Logistic Regression; K-Nearest Neighbors (KNN); Support Vector Machine (SVM); Decision Tree
- **Unsupervised Learning:** K-Means Clustering; Isolation Forest; Principal Component Analysis (PCA)

A suitable model should not only fit the training data but also perform well on unseen data.

Before training, the dataset is typically divided into: Training Set; Testing Set; (Optional) Validation Set. The model learns patterns from the training set, then

makes predictions on the testing set.

Different problems require different evaluation metrics.

- Classification: Confusion Matrix, Accuracy, Precision, Recall, F1-Score, AUC-ROC Curve
- Regression: Mean Absolute Error (MAE); Mean Squared Error (MSE); Root Mean Squared Error (RMSE); R^2 Score
- Unsupervised Models: Silhouette Score, Adjust Rand Index (ARI)

These metrics help determine whether a model is accurate, reliable, and suitable for real-world applications.

> ***Note:*** *ROC Curve and AUC will be introduced in a separate section.*

1. Classification Metrics

In this example, we build a simple **Logistic Regression** model that predicts whether a student will receive a placement offer.

- Generate Dataset

```
import numpy as np
import pandas as pd

np.random.seed(42)

n = 1000

age = np.random.randint(18, 24, n)
gpa = np.round(np.random.uniform(2.0, 4.0, n), 2)

major = np.random.choice(
    ["AI", "CSC", "IT", "SW"],
    n
)

english = np.random.randint(1, 6, n)
```

```
entry_score = np.random.randint(1, 11, n)

# Placement depends mainly on GPA, English and Entry Score
placement = (
    (gpa >= 3.0) &
    (english >= 3) &
    (entry_score >= 6)
)

placement = np.where(placement, "Yes", "No")

df = pd.DataFrame({
    "ID": range(1, n + 1),
    "Age": age,
    "GPA": gpa,
    "Major": major,
    "English Level": english,
    "Entry Score": entry_score,
    "Placement": placement
})

print(df.head(20))
```

- Encode Categorical Variables

```
df2 = df.copy()
df2['Placement'] = df2['Placement'].replace({"Yes": 1, "N
o": 0})
df2 = pd.get_dummies(df2, columns=["Major"], drop_first=Tru
e).replace({True: 1, False: 0})

print(df2.head())
```

- Normalize Numerical Features

```
from sklearn.preprocessing import StandardScaler

X = df2.drop(columns=["ID", "Placement"])
y = df2["Placement"]

scaler = StandardScaler()

X = scaler.fit_transform(X)
```

- Split Training and Testing Sets

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

- Train Logistic Regression

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

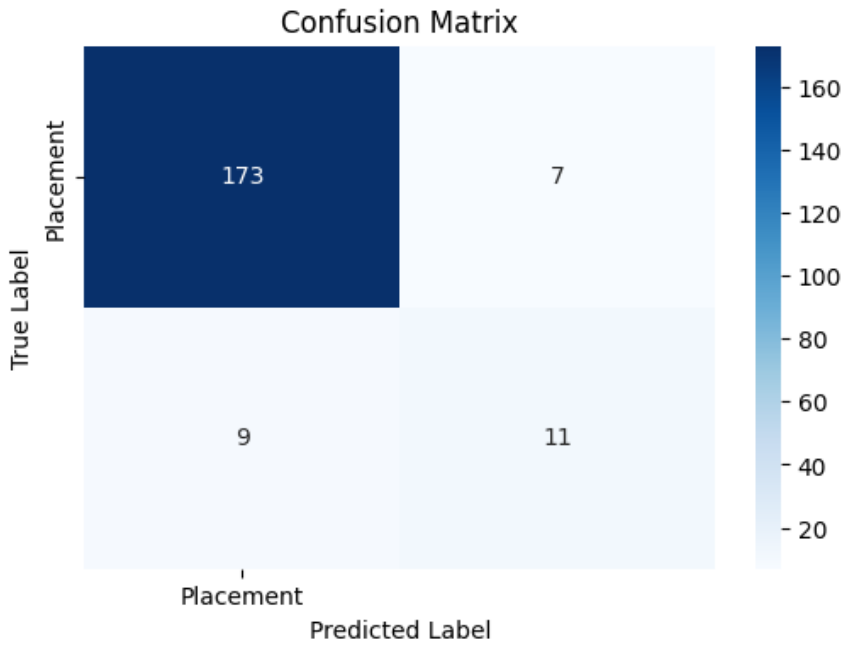
- Confusion Matrix: A confusion matrix summarizes prediction results by counting:
  - **True Positive (TP)** – Correctly predicted positive cases
  - **True Negative (TN)** – Correctly predicted negative cases

- **False Positive (FP)** – Incorrectly predicted positive cases
- **False Negative (FN)** – Missed positive cases

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)
print(cm)
```

```
[[173   7]
 [ 9  11]]
```

```
# Plot confusion matrix
import matplotlib.pyplot as plt
import seaborn as sns

unique_labels = np.unique("Placement").tolist()

plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, cmap="Blues", fmt="d",
            xticklabels=unique_labels, yticklabels=unique_l
abels)

plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix")
plt.show()
```



- **Classification Metrics**
  - **Accuracy** measures the overall proportion of correct predictions.
  - **Precision** measures how many predicted positive cases are actually positive.
  - **Recall** measures how many actual positive cases are correctly identified.
  - **F1-Score** balances Precision and Recall using their harmonic mean.

```
from sklearn.metrics import (accuracy_score, precision_scor
e,
                                                    recall_
score, f1_score)

print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall   :", recall_score(y_test, y_pred))
print("F1 Score :", f1_score(y_test, y_pred))
```

```
Accuracy : 0.92
Precision: 0.6111111111111112
Recall   : 0.55
F1 Score : 0.5789473684210527
```

2. Regression Metrics

In this example, we predict a student's **Weight** from their **Height** using **Linear Regression**. *We also do not split data in this one, primarily to showcase the regression plot.*

- Generate Dataset

```
import numpy as np
import pandas as pd

np.random.seed(42)

n = 200

height = np.random.normal(168, 9, n)

weight = (
    0.52 * height
    - 28
    + np.random.normal(0, 4, n)
)
```

```
df = pd.DataFrame({
    "Height": np.round(height, 1),
    "Weight": np.round(weight, 1)
})

print(df.head(20))
```

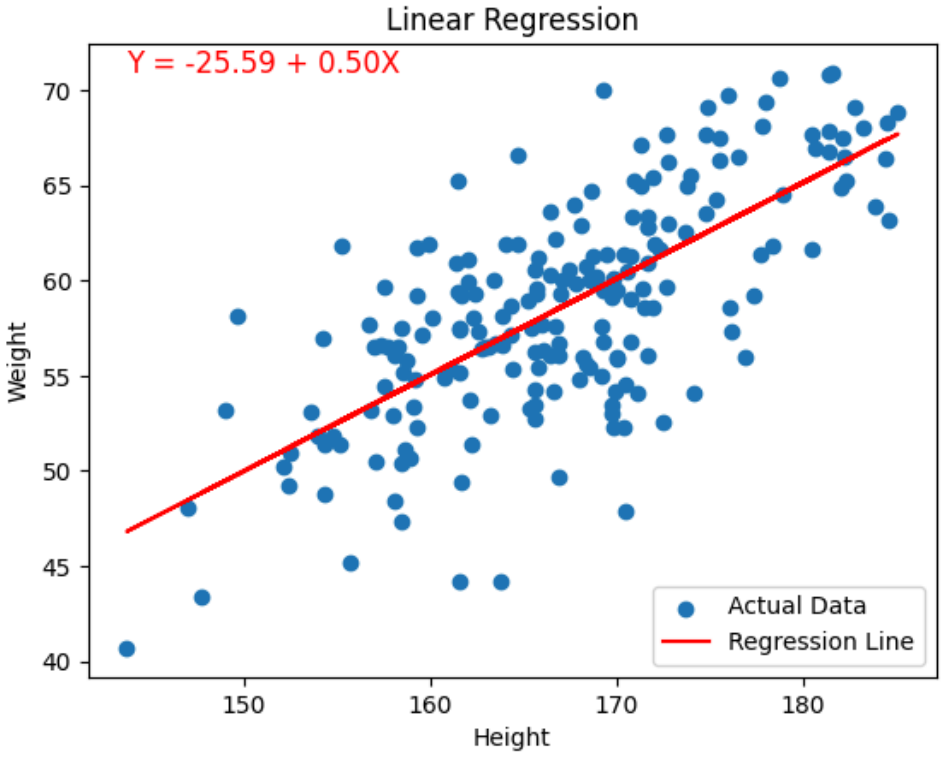
- Train Linear Regression

```
from sklearn.linear_model import LinearRegression

height, weight = height.reshape(-1, 1), weight.reshape(-1,
1)

model = LinearRegression()
model.fit(height, weight)

weight_pred = model.predict(height)
```

- Scatter Plot with Regression Line

```
import matplotlib.pyplot as plt

df["Predicted_Weight"] = weight_pred

# Draw the regression line
plt.scatter(df['Height'], df['Weight'], label='Actual Dat
a')
plt.plot(df['Height'], df['Predicted_Weight'], color='red',
label='Regression Line')
plt.text(x=df['Height'].min(), y=df['Weight'].max(),
         s=f'Y = {model.intercept_[0]:.2f} + {model.coef_
[0][0]:.2f}X', fontsize=12, color='red')
plt.xlabel('Height')
plt.ylabel('Weight')
```

```
plt.legend()
plt.title('Linear Regression')
plt.show()
```



- **Regression Metrics**
  - **Mean Absolute Error (MAE)** measures the average absolute difference between predicted and actual values.
  - **Mean Squared Error (MSE)** penalizes larger prediction errors by squaring the differences.
  - **Root Mean Squared Error (RMSE)** is the square root of MSE and is expressed in the same unit as the target variable.
  - **R^2 Score** indicates how well the model explains the variation in the target variable. A value closer to **1** indicates a better fit.

```
from sklearn.metrics import (mean_absolute_error,
                                                mean_squared_er
ror, r2_score)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("MAE :", mae)
print("MSE :", mse)
print("RMSE:", rmse)
print("R²  :", r2)
```

```
MAE : 0.08
MSE : 0.08
RMSE: 0.282842712474619
R²  : 0.11111111111111127
```

3. Unsupervised Learning Metrics

Unlike classification and regression, **unsupervised learning** does not use labeled target values. Instead, the goal is to discover hidden patterns or group similar observations within the dataset.

One of the most popular clustering algorithms is **K-Means**, which partitions data into a predefined number of clusters based on feature similarity.

Two commonly used evaluation metrics are:

- **Silhouette Score** – Measures how well each sample fits within its assigned cluster.
- **Adjusted Rand Index (ARI)** – Measures the similarity between predicted clusters and known ground-truth labels (if available).

In this example, we cluster students based on their **Height** and **Weight** using K-Means.

- Generate Dataset

```
import numpy as np
import pandas as pd

np.random.seed(42)

# Define total samples (divisible by 3 for equal clusters)
n = 2700
samples_per_cluster = n // 3

# 1. Height for 3 distinct groups (Short, Medium, Tall)
height = np.concatenate([
    np.random.normal(152, 4, samples_per_cluster),  # Clust
er 0
    np.random.normal(168, 4, samples_per_cluster),  # Clust
er 1
    np.random.normal(182, 5, samples_per_cluster)   # Clust
er 2
])

# 2. Weight for 3 distinct groups (Light, Medium, Heavy)
weight = np.concatenate([
    np.random.normal(48, 4, samples_per_cluster),   # Clust
er 0
    np.random.normal(62, 5, samples_per_cluster),   # Clust
er 1
    np.random.normal(78, 6, samples_per_cluster)   # Cluste
r 2
])

# 3. Ground-truth labels updated for 3 clusters (0, 1, and
2)
true_label = np.concatenate([
    np.zeros(samples_per_cluster),                  # 0s
    np.ones(samples_per_cluster),                   # 1s
    np.ones(samples_per_cluster) * 2                # 2s
])

# Create the DataFrame
```

```
df = pd.DataFrame({
    "Height": np.round(height, 1),
    "Weight": np.round(weight, 1),
    "True Cluster": true_label.astype(int)
})

# Display group means to verify separation
df.groupby(by='True Cluster').agg('mean')
```

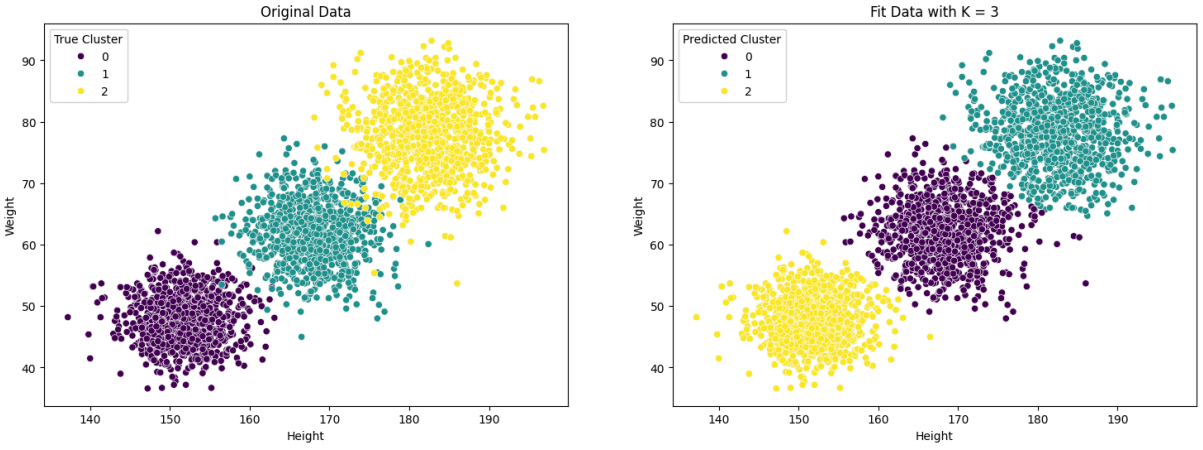
- Train K-Means

```
from sklearn.cluster import KMeans

X = df[["Height", "Weight"]]

kmeans = KMeans(
    n_clusters=3,
    random_state=42
)

pred_cluster = kmeans.fit_predict(X)

df["Predicted Cluster"] = pred_cluster
```

- Visualize the clusters

```
import matplotlib.pyplot as plt

fig, axs = plt.subplots(1, 2, figsize=(18, 6))
sns.scatterplot(data=df, x='Height', y='Weight', hue="True
Cluster",
                palette='viridis', ax=axs[0])

axs[0].set_title('Original Data')
axs[0].set_xlabel('Height')
axs[0].set_ylabel('Weight')

sns.scatterplot(data=df, x='Height', y='Weight', hue="Predi
cted Cluster",
                palette='viridis', ax=axs[1])
axs[1].set_title('Fit Data with K = 3')
axs[1].set_xlabel('Height')
axs[1].set_ylabel('Weight')

plt.show()
```

Depending on the order of the data points or how the clustering algorithm initializes, the assigned cluster colors in the final plots might differ from the ground truth color mapping.



- **Silhouette Score:** The **Silhouette Score** measures how well each sample belongs to its assigned cluster compared to other clusters. Its value ranges from **−1 to 1**.
  - **1** → Well-separated clusters
  - **0** → Overlapping clusters
  - **Negative** → Samples may belong to the wrong cluster

```
from sklearn.metrics import silhouette_score

score = silhouette_score(X, pred_cluster)
print("Silhouette Score:", score)
```

```
Silhouette Score: 0.5979989687536126
```

- Adjusted Rand Index (ARI): The **Adjusted Rand Index (ARI)** compares the predicted clusters with the actual labels. Its value ranges from **−1 to 1**.
  - **1** → Perfect clustering
  - **0** → Random clustering
  - **Negative** → Worse than random

Note: ARI requires known labels and is mainly used for evaluating clustering on benchmark or labeled datasets. In many real-world clustering problems, these labels are unavailable.

```python
from sklearn.metrics import adjusted_rand_score

ari = adjusted_rand_score(
    df["True Cluster"],
    pred_cluster
)

print("Adjusted Rand Index:", ari)
```

```
Adjusted Rand Index: 0.9487555275993816
```

Summary

In this section, we explored the most common evaluation metrics used in supervised and unsupervised machine learning.

- For **classification**, we trained a **Logistic Regression** model and evaluated its performance using the **Confusion Matrix**, **Accuracy**, **Precision**, **Recall**, and **F1-Score**. These metrics measure how effectively the model classifies different categories.
- For **regression**, we trained a **Linear Regression** model and assessed its prediction quality using **Mean Absolute Error (MAE)**, **Mean Squared Error (MSE)**, **Root Mean Squared Error (RMSE)**, and the **R^2 Score**. These metrics quantify prediction errors and indicate how well the model fits the observed data.
- For **unsupervised learning**, we applied **K-Means Clustering** to group similar observations based on height and weight. We then evaluated the clustering results using the **Silhouette Score**, which measures cluster cohesion and separation, and the **Adjusted Rand Index (ARI)**, which compares predicted clusters with known labels when ground-truth information is available.

Different machine learning tasks require different evaluation metrics. Selecting appropriate metrics is essential for objectively measuring model performance, comparing algorithms, and ensuring that models generalize well to unseen data.

The later section will introduce the **ROC Curve** and **Area Under the Curve (AUC)**, which provide additional insights into the performance of classification models.

References

https://www.ibm.com/think/topics/model-training

https://www.geeksforgeeks.org/machine-learning/metrics-for-machine-learning-model/

Next Section:

10. Feature Selection